Outlook Slack Bot Overview & Development Paper:

CUI-accredited AWS GovCloud enabler bridging Slack, Microsoft Outlook, and a Foundry-governed LLM gateway

Current as Of: July 2026

[Names, the originating organization, and internal network/account identifiers have been redacted from this version for external sharing.]

Table of Contents

Executive Summary	3
Bot Technical Overview	3
Considerations	6
Way Forward	6

Executive Summary:

The Outlook Slack Bot is an intelligent, internal client that unifies Outlook and Slack to eliminate routine logistical overhead. Through a natural-language Slack DM, a user can prompt a governed internal LLM to draft email replies directly in Outlook as well as summarize, analyze, or query the user's inbox and Slack channels.

What makes the system notable is not the specific feature set later described, but the operating envelope and ease of use from a user perspective. The bot runs entirely inside a CUI-accredited AWS GovCloud boundary, uses Microsoft Graph for US Government (graph.microsoft.us), and routes every LLM call through an internally governed Foundry gateway rather than a public API endpoint. It has no inbound network access, no SSH, no public IP, a code-enforced data boundary, and a fail-closed control that prevents Controlled Unclassified Information (CUI) from reaching Slack. The user has access to this boundary through a single interface via a message to the Slack application.

This paper describes the complete system as it stands, the connections between its parts, the security model, and the sequence of engineering decisions and fixes that produced it.

Bot Technical Overview:

The following will detail the architecture, security model, and components which enable the below core capabilities of the bot:

Bot Core Capabilities:

- **Drafts Outlook replies in the user's tone from Slack:** Through a DM to a Slack application, a user requests an email be drafted, Slack confirms the details of the draft, and the bot saves the email as a draft in the user's Outlook Drafts, and never sends the email itself (send is not an allowed permission in Graph).
- **Drafts to multiple recipients at once:** A single request (“Personalize draft invitations to these 20 emails”) produces a personalized draft per recipient, all previewed and saved together (with subsequent editing or saving requests).
- **Provides analysis and answers questions on the user's mailbox:** In a Slack DM, the user asks about their inbox; the bot performs a live Microsoft Graph search of their Inbox and Sent Items and answers with cited results.
- **Summarize and analyze Slack channels:** After adding the bot to a Slack channel, “summarize #channel” in the application DM returns an analytical, domain-aware focused summary and analysis depending on the user prompt.
- **Answers predicated on organization- and industry-aware context:** Every run reads an accurate, curated internal context block.
- **Enforces CUI egress control:** Any email that is CUI-marked, CUI-labeled, or encrypted-to-external domains is withheld from Slack and the user is informed it is withheld from Slack.

The bot is able to accomplish the above through four established endpoints, each with narrow, specified egress paths:

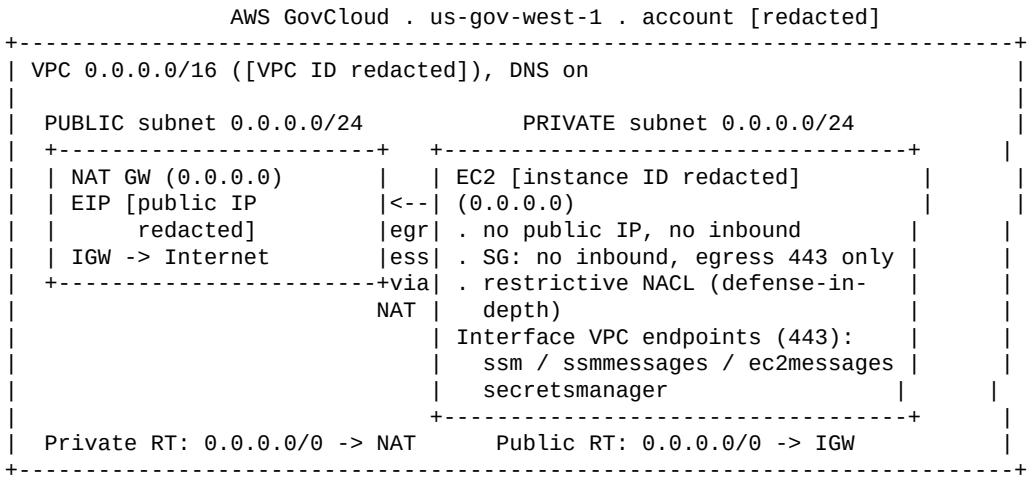
Bot Egress Endpoints:

Connection	Endpoint	Purpose	Authentication
Slack	Slack Web API	Read Slack DMs, post replies, read Slack channels	Bot User OAuth token (xoxb-), scoped for this use case
Microsoft Outlook	graph.microsoft.us	Search mailbox, create drafts	Azure AD app, client credentials
LLM (Foundry Gateway)	[internal gateway host – redacted]	All calls to the model (summaries, drafts, user profile)	Foundry gateway bearer token
AWS Secrets Manager	Private VPC endpoint	Fetch all runtime config/secrets	IAM role for the EC2 instance
(future add) internal data platform	[internal subdomain – redacted]	Search internal records	TBD

The LLM connection very specifically, is not exposed to any public AI provider. llm_client.py hard-codes an allow-list of the Foundry gateway host and refuses in code to call anything else (with an explicit block on anthropic.com).

A purpose-built VPC runs the bot on low compute using a single t3.micro (~2x vCPU, 1 GB RAM) EC2 instance in AWS. An architectural trade-off in choosing not to employ an MCP server is that we do not require a persistent daemon. The bot instead is a poller, which uses a systemd timer + oneshot service meaning, every ~5 seconds the timer fires slack_poll.py, which drains any new Slack DMs, answers them, and exits. By nature of not maintaining persistent memory, security is baked in — there is nothing to leak or wedge. There are multiple levels of data protection and security in the bot's architecture.

Bot High Level Network Architecture:



A user call flows email content through only: Graph US Gov → the bot → Foundry gateway. The LLM is forced to the Foundry gateway in code through a local squid proxy (host allow-list) and in the network via an iptables owner match (egress lockdown / no direct internet path). Secrets are also specifically fetched by AWS SSM over a private VPC endpoint, not the open internet. Internally, the bot will never answer in a channel via an explicit guard, preventing any messages regarding another user's Outlook from being leaked in a main channel. A Slack user can never prompt another user's mailbox through the specific mapping in users.json. There is no default mailbox if the Slack user ID is not mapped — the call to read a mailbox will fail and inform the user. Read and write privileges to a mailbox are also specifically governed by the Exchange Application Access Policy in Graph; it is not flexible code. There is a CUI filter which fails a return from an email with CUI markings in the subject/body or if there are encryptions to an external email domain. Lastly, requirements.txt is pinned and hashed on the Linux platform so a PyPI artifact with any malicious code cannot be substituted.

After adequate security measures were implemented, feature enhancement* was worked through beta testing with the bot.

Module	Role
`slack_poll.py*`	Main poller: reads DMs, orchestrates the answer/draft/summarize flows, posts replies. “On it, one moment” while waiting to get responses.
`llm_client.py`	The only LLM path; Foundry-gateway host allow-list (data boundary)
`tools.py`	The agentic tool loop's tools: search_emails, draft_email, read_slack_channel
`graph_search.py`	Live Graph mailbox search + the CUI filter chokepoint
`cui_filter.py`	Fail-closed CUI detector
`channel_summary.py*`	Reusable, gateway-routed Slack channel summarizer
`org_context.py*`	Accurate internal company/product facts, shared by summaries + Q&A;
`create_draft.py*`	Creates Outlook drafts via Graph
`voice_profile.py / build_voice_profile.py*`	Per-user writing-voice profiles from Sent mail
`user_map.py / users.json`	Slack-user → mailbox authorization map
`config.py`	Secrets Manager / .env resolution
`state_store.py`	Per-DM conversation state (local JSON or DynamoDB)

`deploy/`	Idempotent deploy + teardown scripts, user-data, squid/iptables config

Each module marked with an asterisk was edited through prompt testing and bot answer evaluation. Each feature build was focused around ease of use for the user: accepting a wider range of prompts from a user, creating email drafts closer to the user's voice profile, reading and summarizing channels in Slack, etc. Before handoff was completed with the new bot POC, all the updated scripts from the bot on the local server were synced and pushed to an internal enterprise GitHub repository [URL redacted].

Considerations:

The bot began as a datasource -> object -> workshop application in Foundry.

[Two screenshots of the Foundry dataset flow and workshop view were omitted from this version.]

In order to realize the Slack integration API, the initial code was migrated off the Foundry platform (via a Claude-based model) to a local server. During this migration multiple implementation trades were assessed: MCP server, Claude @tag, or the current local-server poller.py. Pros and cons for each are documented in an internal wiki page [link redacted]. Largely due to the data boundary constraints, the self-hosted poller implementation was selected as most advantageous for the organization. Without the presence of CUI data, the ability to utilize commercial Claude and the Anthropic Tag SaaS would greatly speed up development time (to nearly none) and reduce effort to adopt.

Way Forward:

Conducted final handoff on 22 July 2026. The plan moving forward is to add an internal data platform as a fifth external connection and data source. This additional integration would allow for organizational data from internal systems (inventory, nonconformance reports, material planning, CRM/sales records, etc.) to be queryable from Slack. Enabling sales to find deal information across Outlook and internal CRM/sales records in under 30 seconds through a single, user-friendly interface. This integration would not impose any architectural changes as the bot lives on EC2.

The bot as is on the EC2 instance is portable. Once the repo is cloned, the AWS path (package → S3 → SSM pull+unzip+restart) is configured to autodiscover the AWS path by tag in $AWS_BIN. The operator will need credentialed access to Git/internal GHE, AWS CLI authenticated to GovCloud (account [redacted]) with S3 + ssm:SendCommand + ec2:DescribeInstances, and Python 3. In the future, it would be worth exploring the establishment of an operational CI/CD pipeline so a main code push triggers deployment directly into the EC2 instance via the GHE. Right now the GHE serves primarily as version control, which is not the cleanest path architecturally.

All in all, a very enlightening project to work on!